

HAMi COMMUNITY WEBINAR

HAMi 2.9 如何重构 Kubernetes AI 算力调度

HAMi 2.9 社区技术直播
Live Demo + Maintainer Q&A

李孟轩

HAMi Maintainer/密瓜智能联合创始人兼CTO

2026.05.14



AGENDA

议程

01

HAMi 新版本发布 (v2.9.0/v2.8.3)

02

HAMi 新版本重点功能介绍

03

HAMi DRA

04

Q&A

SECTION 1

HAMi 新版本发布 (v2.9.0/v2.8.3)

如何更新，部署，需要注意的事项

更新方式：helm upgrade/helm uninstall + helm install

需要注意的事项：

- 一定要更新Chart，不能只替换镜像，否则会导致更新失败
- 从2.5.0之后，升级HAMi不会导致任务失败

SECTION 2

HAMi 新版本重点功能介绍

HAMi v2.9新增了哪些重点功能，如何使用

特性1: HAMi-Core优化 (v2.8.3/v2.9.0共同特性)

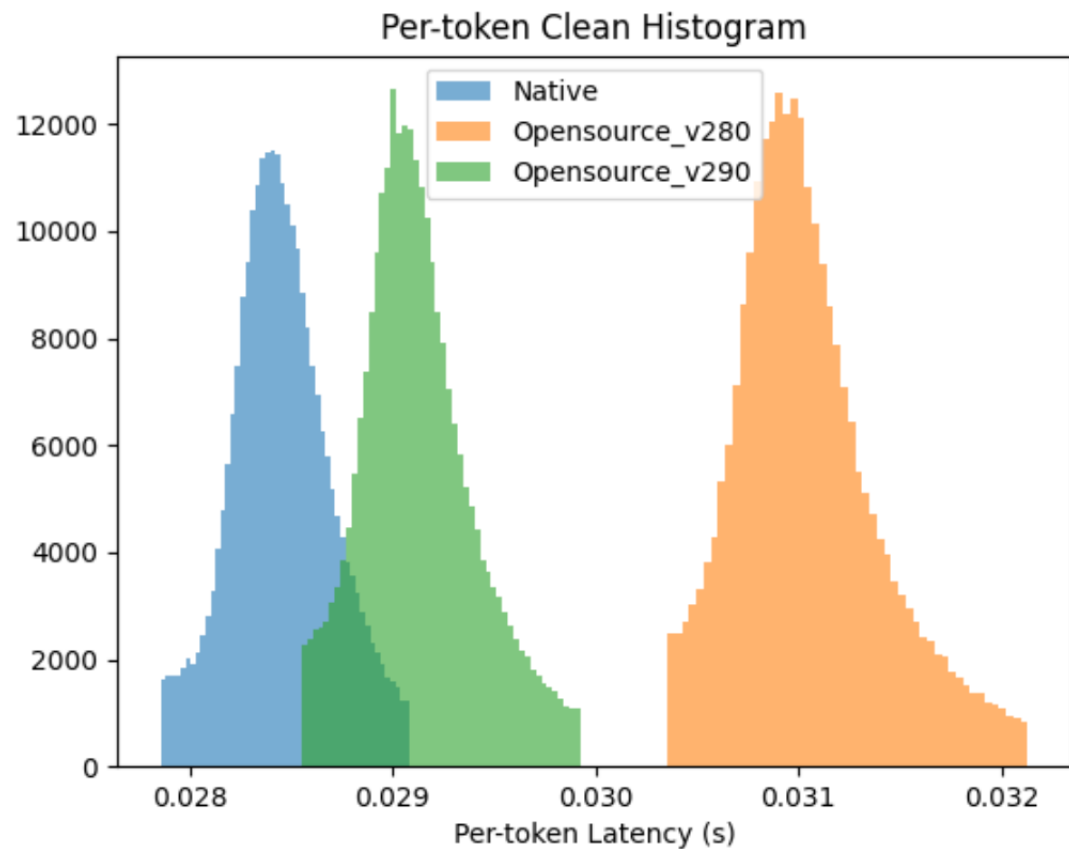
问题修复:

vllm v0.18+版本在运行多卡任务会导致异常退出

性能优化:

优化了HAMi-Core的消息同步机制, 使推理性能显著提升

特性1: HAMi-Core优化 (v2.8.3/v2.9.0共同特性)



Test Cases:

test id	case	type	params
6.1	Qwen3-8B (vLLM)	inference	batch=1, stream=True, max_model_len=8192

Test Result:

Metric	Native	Opensource_v280	Opensource_v290
TTFT p50 (s)	0.0621	0.0670	0.0629
TTFT p95 (s)	0.0642	0.0713	0.0650
TTFT p99 (s)	0.0652	0.0735	0.0674
Per-token latency (clean mean, s)	0.0285	0.0310	0.0291

特性2: 昇腾软切分方案



使用场景: 中小模型微调、在线推理、CI/CD 测试任务需细粒度配额

特性2: 昇腾软切分方案



1 用户态拦截层 (libvnpu.so)

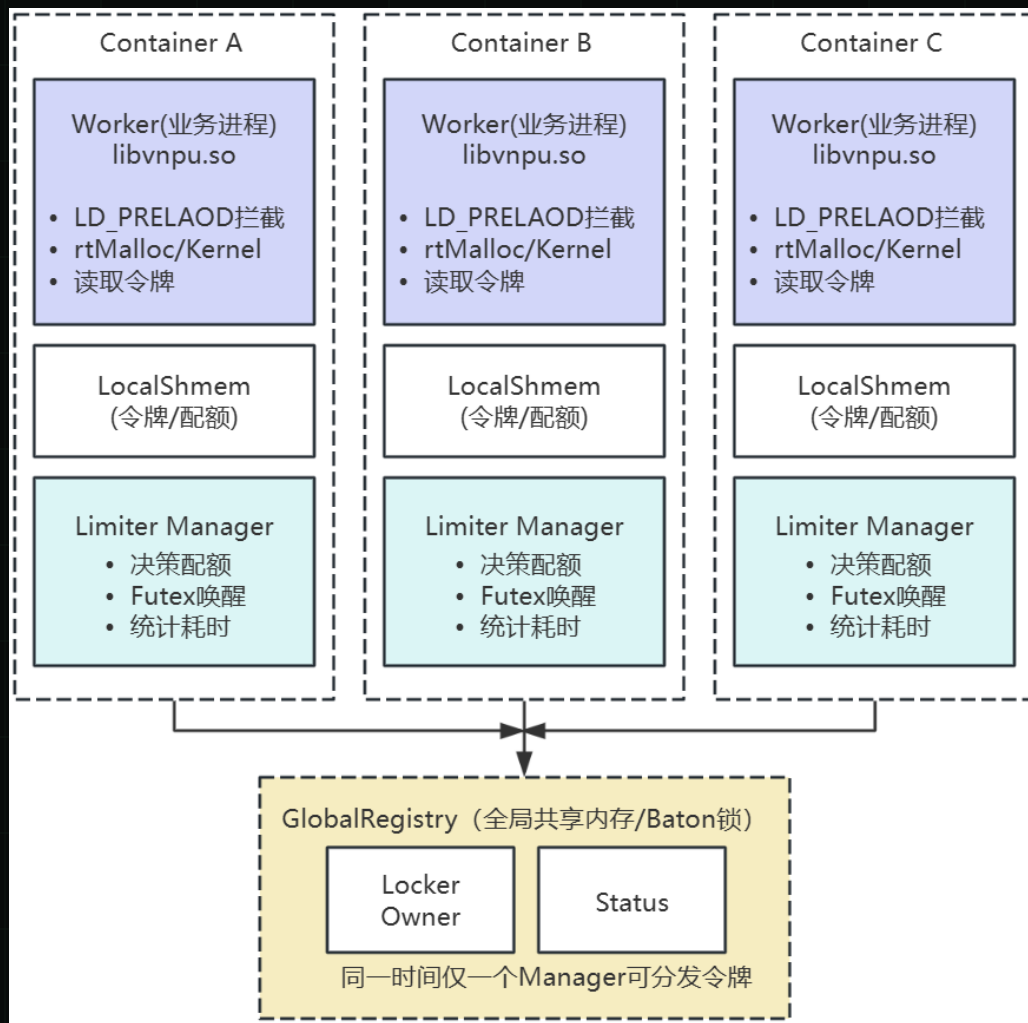
- 通过LD_PRELOAD注入用户进程
- 拦截NPU Runtime API (rtMalloc/rtKernelLaunch)
- 读取共享内存中的令牌和配额, 检查通过放行执行

2 容器内管理层 (Limiter Daemon)

- 独立Daemon进程, 创建LocalContainerShmem
- 定期计算令牌配额并写入共享内存
- 通过Futex唤醒等待的Worker线程
- 报告任务耗时并统计执行效率

3 全局协调层 (GlobalRegistry)

- 容器间limiter通过全局共享内存协调
- Baton锁机制轮转调度权, 同一时间仅一个容器的Manager可分发令牌
- 实现跨容器互斥与公平调度



特性2: 昇腾软切分方案

```

lifecycle:
  postStart:
    exec:
      command:
        - "bash"
        - "-c"
        - |
          export RUST_LOG=info
          /hami-vnpu-core/limiter > /hami-vnpu-core/inst1_manager.log 2>&1
  
```

```

func (ps *PluginServer) Allocate(ctx context.Context, reqs *v1beta1.AllocateRequest)
/*
1. (可省略)处理设备注入 可见设备环境变量设置后 设备文件会自动注入)
2. 处理存储卷注入 (-v)
   A. 驱动与 SMI 工具链
   B. vnpu-core代码路径 : /usr/local/hami-vnpu-core
   C. 注入 HAMi 劫持库路径 (对应脚本中的 LIBRARY_PATH), 通过/etc/ld.so.preload 挂载
   D. HAMi 算力切分专用共享目录 : /usr/local/hami-shared-region:/hami-shared-region
3. 环境变量注入
   A. 可见设备IDs
   B. 共享内存通信路径: GLOBAL_SHM_PATH = /hami-shared-region/{ID}_global_registry
   C. 显存配额MEM_QUOTA: 从Annotation读取 memory = 28672
   D. 优先级PRIORITY: 从Annotation读取 core = 20
*/
*/
}
  
```

MutateAdmission阶段自动注入
lifecycle.postStart,
启动 limiter 进程

自动创建所需目录并设置权限
宿主机部署+动态链接劫持



PostStart 钩子注入



安全就绪检测



libvnpu.so 启动时循环检测
/dev/shm/ 令牌文件



注入关键存储卷
注入环境变量 (显存、算力)

特性2: 昇腾软切分方案配置方式

部署时配置ascend.hamiVnpuCore: true即可开启 (全局)

在ascend-device-node-config中进行配置(节点级)

```
ascend:
  enabled: false
  image: ""
  imagePullPolicy: IfNotPresent
  extraArgs: []
  nodeSelector:
    ascend: "on"
  tolerations: []
  runtimeClassName: "" # Name of the runtime class to set
  hamivnpuCore: false # When true, all nodes will enable soft partitioning
  customresources:
    - huawei.com/Ascend910A
    - huawei.com/Ascend910A-memory
    - huawei.com/Ascend910A-core
    - huawei.com/Ascend910B2
    - huawei.com/Ascend910B2-memory
    - huawei.com/Ascend910B2-core
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  labels:
    app.kubernetes.io/component: hami-scheduler
    app.kubernetes.io/name: hami
    app.kubernetes.io/instance: hami
  name: hami-device-node-config
  namespace: kube-system
data:
  node-config.yaml: |-
    nodes:
      - name: "cnst-dev-w2"
        hamivnpuCore: true
        vDeviceCount: 8
```

特性2: 昇腾软切分方案使用方式

当前版本需要在annotations中进行声明, 未进行声明的任务将会使用传统vNPU方式分配

```
apiVersion: v1
kind: Pod
metadata:
  name: vllm-npu-2card
  annotations:
    huawei.com/vnpu-mode: 'hami-core' # Enable hami-vnpu-core soft partitioning
spec:
  containers:
    - name: vllm-container
      image: vllm-ascend:latest
      command: ["/bin/sh", "-c"]
      args:
        - |
          vllm serve /model/Qwen3-0.6B \
            --host 0.0.0.0 \
            --port 8002 \
            --enforce-eager \
            --tensor-parallel-size 2 \
            --gpu-memory-utilization 0.5 # Key parameter: Total requested memory 64Gi / Total p
resources:
  limits:
    huawei.com/Ascend910B3: "2" # Request 2 virtual devices for parallel comput
    huawei.com/Ascend910B3-memory: "65536" # Total memory limit for the container (64GiB
    huawei.com/Ascend910B3-core: "50"
```

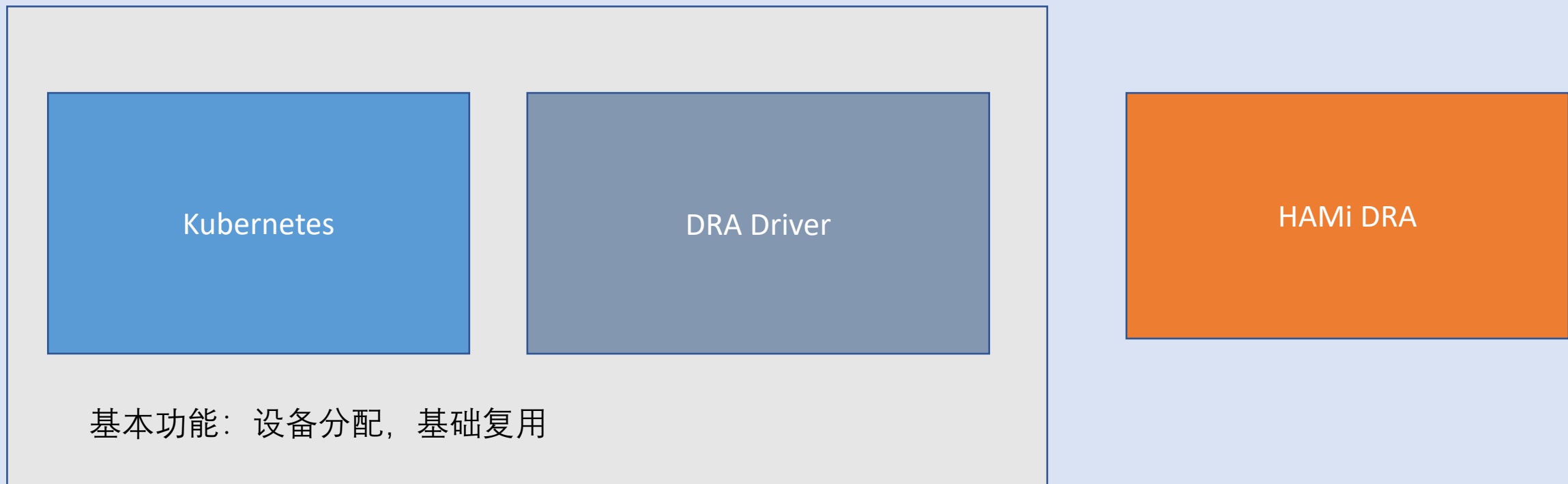
昇腾软切分方案演示

SECTION 3

HAMi-DRA Ready for use!

HAMi v2.9新增了哪些重点功能，如何使用

为什么我们需要HAMi-DRA（社区视角）



基本功能：设备分配，基础复用

最佳体验：用户无感知，集群监控，调度事件

为什么我们需要HAMi-DRA（用户视角）



HAMi的部署包含一个调度器和对应设备的device-plugin，会与其它未适配的调度方案冲突，使用户被迫面临二选一的问题

如何部署HAMi-DRA

项目github

```
https://github.com/Project-HAMi/HAMi-DRA
```

使用Helm一键部署

```
> git clone https://github.com/Project-HAMi/HAMi-DRA.git  
> helm install hami-dra ./charts/hami-dra
```

使用方式:

部署之后指定资源名即可

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: gpu-pod  
spec:  
  containers:  
    - name: ubuntu-container  
      image: ubuntu:18.04  
      command: ["bash", "-c", "sleep 86400"]  
      resources:  
        limits:  
          nvidia.com/gpu: 2 # 请求 2 个 vGPU  
          nvidia.com/gpumem: 3000 # 每个 vGPU 请求 3G 设备显存
```

HAMi-DRA 方案演示

HAMi-DRA 使用场景总结

标准化的集群，要求最优调度的，使用HAMi

高度定制化的集群，希望即插即用，并兼容已有的定制化组件的，使用HAMi-DRA

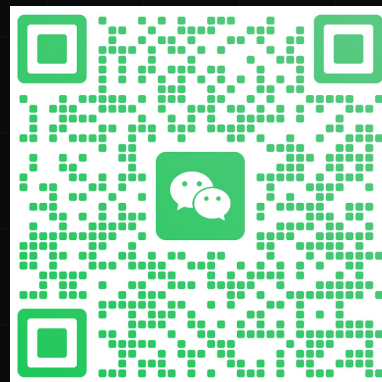
SECTION 4

Questions & Answers

谢谢观看

欢迎提问 / 加入社区 / 关注项目

Thank you



扫码加入 HAMi 社区